

```

// Headless verification of Demo 29's math (Theorem 6b, D8)
const Zv=150, Zh=250, D=Zh-Zv;
const rayAt=(a,z)=>{const l=(z-Zv)/D; return [a[1]*l, a[0]*(1-l), z];};
const rayDir=(a)=>[a[1],-a[0],D];
const footl=(a)=>[0,a[0],Zv];
const cr=(A,B,C,Dd)=>((C-A)*(Dd-B))/((C-B)*(Dd-A));
const cross=(u,v)=>[u[1]*v[2]-u[2]*v[1],u[2]*v[0]-u[0]*v[2],u[0]*v[1]-u[1]*v[0]];
const sub=(u,v)=>[u[0]-v[0],u[1]-v[1],u[2]-v[2]];
function fit3(pts){
  const R=pts.map(([s,t])=>[s,1,-s*t,-t]);
  const m=(c)=>R[0][c[0]]*(R[1][c[1]]*R[2][c[2]]-R[1][c[2]]*R[2][c[1]])
    -R[0][c[1]]*(R[1][c[0]]*R[2][c[2]]-R[1][c[2]]*R[2][c[0]])
    +R[0][c[2]]*(R[1][c[0]]*R[2][c[1]]-R[1][c[1]]*R[2][c[0]]);
  return {a:m([1,2,3]),b:-m([0,2,3]),c:m([0,1,3]),d:-m([0,1,2])};
}
const mob=(M,s)=>(M.a*s+M.b)/(M.c*s+M.d);
function transversalAt(u,R1,R2,R3){
  const p=rayAt(R1,u);
  const n2=cross(sub(rayAt(R2,Zv),p),rayDir(R2));
  const n3=cross(sub(rayAt(R3,Zv),p),rayDir(R3));
  const d=cross(n2,n3), n=Math.hypot(...d);
  return n<1e-9?null:{p,d:[d[0]/n,d[1]/n,d[2]/n]};
}
function lineRayDist(L,a){
  const q=footl(a), v=rayDir(a), c=cross(L.d,v), n=Math.hypot(...c);
  const w=sub(q,L.p);
  return n<1e-9?Math.hypot(...cross(w,L.d))
    :Math.abs(w[0]*c[0]+w[1]*c[1]+w[2]*c[2])/n;
}
const rnd=()=> (Math.random()-0.5)*300;

// 1) forward: t_i = phi(s_i) for a random Mobius phi -> lambda == mu
let w=0;
for(let k=0;k<5000;k++){
  const a=rnd(),b=rnd(),c=(Math.random()-0.5)*2,d=rnd()/3+80;
  const phi=(s)=>(a*s+b)/(c*s+d);
  const S=[rnd(),rnd(),rnd(),rnd()];
  if(new Set(S.map(x=>x.toFixed(3))).size<4) continue;
  const T=S.map(phi);
  if(T.some(t=>!isFinite(t)||Math.abs(t)>1e5)) continue;
  const lam=cr(S[0],S[1],S[2],S[3]), mu=cr(T[0],T[1],T[2],T[3]);
  w=Math.max(w, Math.abs(lam-mu)/(1+Math.abs(lam)));
}
console.log("Thm 6b forward: |lam-mu| rel      :", w.toExponential(2), "(doc: 1e-9

```

```

// 2) converse: perturb t4 off the chain -> lambda != mu
const S=[-80,30,90,-20], M=fit3([[S[0],-60],[S[1],20],[S[2],100]]);
const T=[ -60, 20, 100, mob(M,S[3]) ];
const lamOn=cr(...S), muOn=cr(...T);
const muOff=cr(T[0],T[1],T[2],T[3]+5);
console.log("on-chain lam,mu          :", lamOn.toFixed(9), muOn.toFixed(9));
console.log("perturbed mu (separates)  :", muOff.toFixed(9));

// 3) co-chained 4-tuple: EVERY transversal of r1r2r3 also meets r4 (Thm 4)
const RAYS=[[S[0],T[0]],[S[1],T[1]],[S[2],T[2]],[S[3],T[3]]];
let wg=0;
for(let u=-200;u<=600;u+=25){
  const L=transversalAt(u,RAYS[0],RAYS[1],RAYS[2]);
  if(!L) continue;
  for(const r of RAYS) wg=Math.max(wg, lineRayDist(L,r));
}
console.log("regulus sweep: max gap to all 4  :", wg.toExponential(2));

// 4) off-chain criterion is the MAX gap: a generic line stabs the quadric
//      in 2 points, so 2 members always touch (min gap ~ 0 is EXPECTED);
//      but some member must miss badly unless r4 lies in the surface.
const R4off=[S[3], T[3]+20];
let mn=1e18, mx=0;
for(let u=-220;u<=640;u+=(640+220)/12){
  const L=transversalAt(u,RAYS[0],RAYS[1],RAYS[2]);
  if(!L) continue;
  const d=lineRayDist(L,R4off);
  mn=Math.min(mn,d); mx=Math.max(mx,d);
}
console.log("off-chain: min gap (2 stabs, ~0) :", mn.toFixed(3));
console.log("off-chain: MAX gap (criterion)   :", mx.toFixed(3), "(>> 0)");

// 5) bi-harmonic preset: lambda = mu = -1
const bh=[[-80,-60],[40,80],[0,50],[160,132.5]];
console.log("preset lam, mu          :",
  cr(bh[0][0],bh[1][0],bh[2][0],bh[3][0]).toFixed(9),
  cr(bh[0][1],bh[1][1],bh[2][1],bh[3][1]).toFixed(9));

```